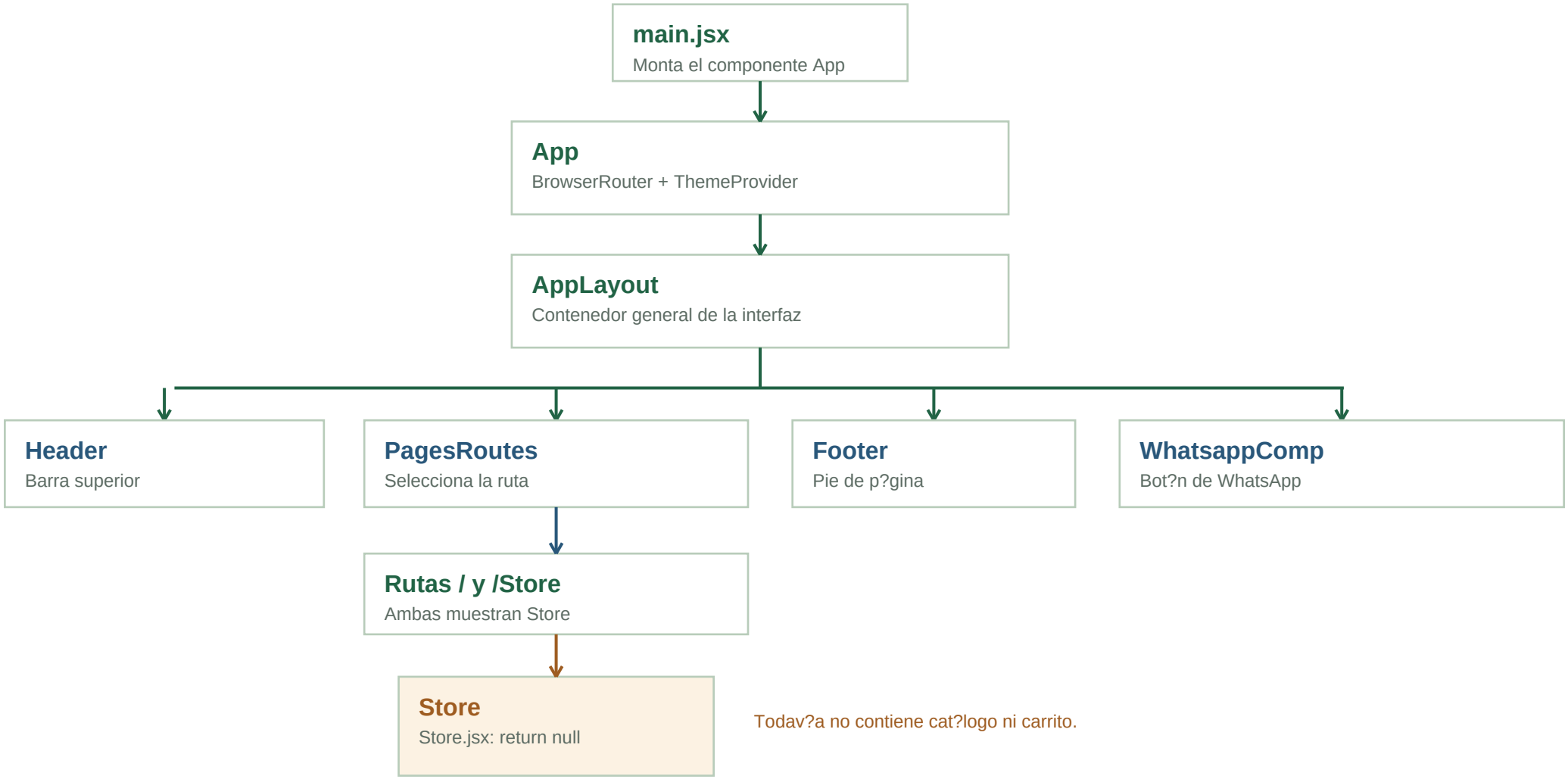
CROQUIS DE COMPONENTES

La tienda: estructura actual

Qu? renderiza la aplicaci?n hoy y qu? piezas siguen desconectadas.

?RBOL ACTUAL DE LA APLICACI?N



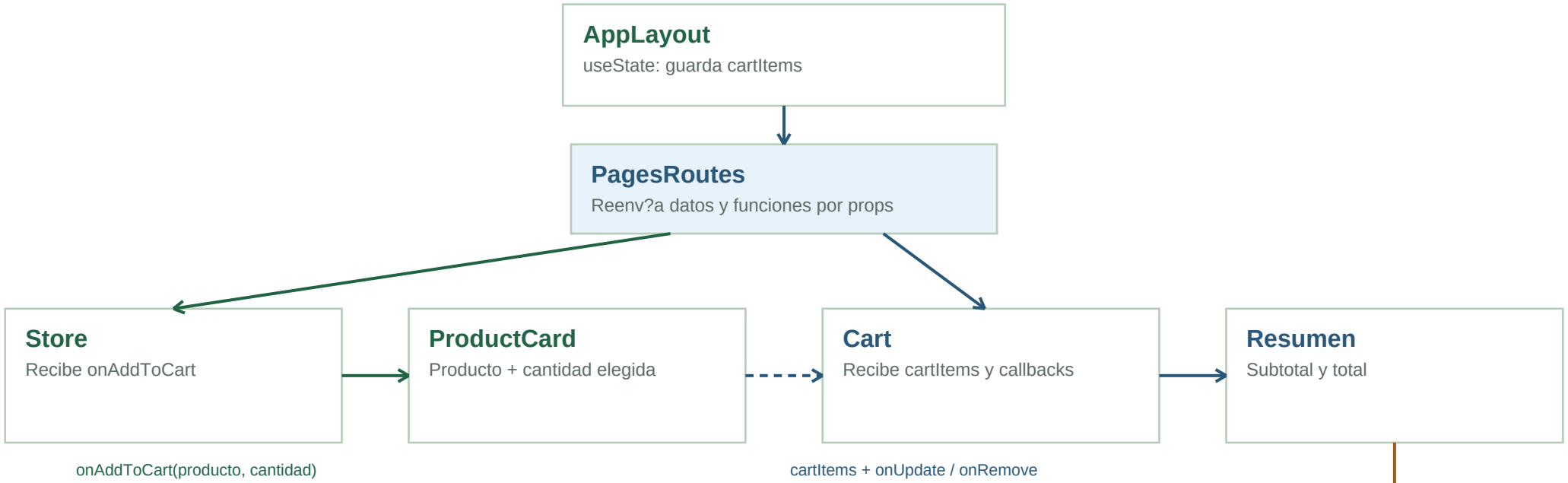
PROTOTIPO APARTE (NO LO RENDERIZA Store)

Storecomp/Card y Storecomp/Counter tambi?n son esqueletos sin conexi?n.

PROPUESTA SIN CONTEXT

Carrito con props y estado compartido

No hace falta useContext: el padre guarda el carrito con useState y reparte props.



C?MO VIAJAN LAS PROPS

1. AppLayout mantiene el carrito en su estado (useState).
2. Pasa cartItems y funciones a PagesRoutes como props.
3. PagesRoutes entrega las props a Store y Cart en cada ruta.
4. ProductCard llama onAddToCart; Cart llama editar / quitar.

El estado vive arriba; las pantallas reciben datos y callbacks.

Flujo: elegir en Store -> revisar en Cart -> confirmar -> ver el ticket.

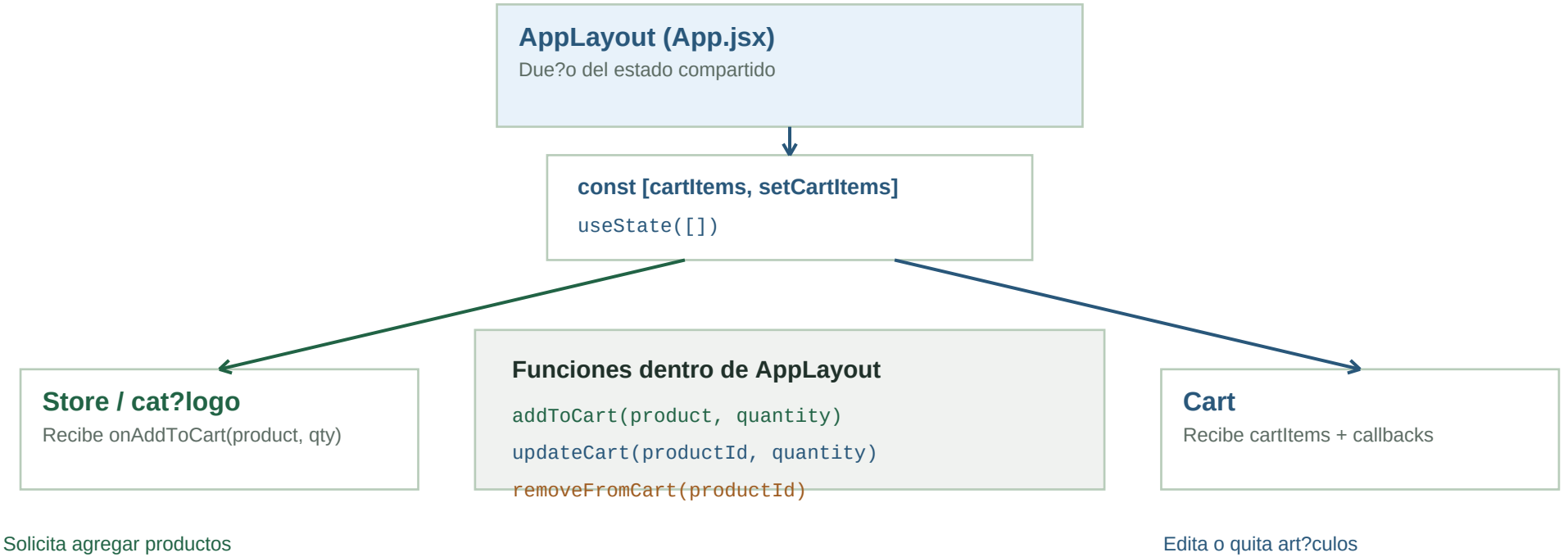
Esta estructura es una propuesta; hoy Store y Cart siguen vac?os.

M?s adelante se puede usar Context si se aprende y hace falta.

PROPUESTA DE GESTI?N DE COMPRA

?D?nde vive el carrito?

El estado y sus operaciones viven en AppLayout, que permanece al cambiar de ruta.



?Por qu? en el componente padre?

AppLayout contiene PagesRoutes y no se desmonta al navegar entre Store y Cart.

As? ambos ven el mismo cartItems; Store y Cart no necesitan duplicar el estado.

PagesRoutes no crea el carrito: solo reenv?a por props los datos y las funciones.

No usamos useContext para esta propuesta; solo useState y props.

En resumen: AppLayout administra; Store pide cambios; Cart muestra y actualiza la selecci?n.

Estas funciones a?n no existen en el c?digo actual: son la ubicaci?n propuesta.

DETALLE DE LAS OPERACIONES

Qu? hacen add, update y remove

Cada cambio crea un nuevo array; el actualizador funcional usa siempre el estado m?s reciente.

1. Agregar: sumar cantidad si ya existe; si no, crear una l?nea de carrito.

```
function addToCart(product, quantity = 1) {
  setCartItems((items) => {
    const existing = items.find((item) => item.id === product.id)
    if (existing) {
      return items.map((item) =>
        item.id === product.id
          ? { ...item, quantity: item.quantity + quantity }
          : item
      )
    }
    return [...items, { ...product, quantity }]
  })
}
```

2. Actualizar cantidad: cambiar la cantidad de una l?nea existente.

```
function updateCart(productId, quantity) {
  setCartItems((items) => items.map((item) =>
    item.id === productId ? { ...item, quantity } : item
  ))
}
```

3. Quitar: eliminar del array la l?nea cuyo id coincide.

```
function removeFromCart(productId) {
  setCartItems((items) => items.filter((item) => item.id !== productId))
}
```

Contrato de datos: cada l?nea conserva campos del producto (id, title, price) y quantity.

La interfaz deber?a limitar quantity a enteros mayores que cero; quitar usa su propia acci?n.

addToCart acepta la cantidad seleccionada por el usuario; si no se indica, agrega 1 unidad.

DE RUTA A COMPROBANTE

C?mo pasan las props y cu?ndo nace el ticket

PagesRoutes elige la pantalla; el pedido se confirma desde Cart y conserva el precio de compra.

AppLayout: define el estado y las funciones; entrega las props a PagesRoutes.

```
<PagesRoutes
  cartItems={cartItems}
  onAddToCart={addToCart}
  onUpdateCart={updateCart}
  onRemoveFromCart={removeFromCart}
  onCheckout={checkout}
/>
```

PagesRoutes: pasa a cada p?gina lo que necesita.

```
<Route path="/" element={
  <Store onAddToCart={onAddToCart} />
} />
<Route path="/Cart" element={
  <Cart cartItems={cartItems}
    onUpdateCart={onUpdateCart}
    onRemoveFromCart={onRemoveFromCart}
    onCheckout={onCheckout} />
} />
```

?En qu? momento se genera el ticket?

Al presionar ?Finalizar compra? en Cart se crea/guarda un pedido con sus l?neas, cantidades, precios unitarios aplicados y total. El ticket representa ese pedido confirmado; **el carrito solo representa la selecci?n que todav?a se est? revisando.**

Responsabilidades en el caso de uso

1. Store: cat?logo -> ProductCard permite elegir cantidad y agregar al carrito.
2. Cart: muestra l?neas, permite actualizar/quitar, calcula subtotal y total.
3. Confirmar: registra el pedido; Ticket/Comprobante muestra el detalle final.
4. El pedido guarda una copia de precios/nombres para que tickets previos no cambien.

Hoy Store.jsx y Cart.jsx est?n vac?os; Storecomp/Card y Counter tambi?n son esqueletos.

La propuesta no est? implementada a?n; el prototipo ProductosDiet est? desconectado de Store.